

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший разработчик в X5 Tech

_____ В. Е. Козловский
 «__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
 образовательной программы
 "Программная инженерия", старший
 преподаватель департамента
 программной инженерии

_____ Н. А. Павлочев
 «__» _____ 2026 г.

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
 ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Руководство оператора

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.09 34 01-1-ЛУ

Исполнители:

Студент группы БПИ235

_____ / А. К. Соколова /
 «__» _____ 2026 г.

Студент группы БПИ236

_____ / С. Е. Растворов /
 «__» _____ 2026 г.

Студент группы БПИ244

_____ / Л. А. Субботин /
 «__» _____ 2026 г.

Инва.№ подп	Подп. и дата	Взам. инв.№	Инва.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.05.09 34 01-1-ЛЮ

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Руководство оператора

RU.17701729.05.09 34 01-1

Листов 9

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

Исполнители:

Студент группы БПИ235

_____ / А. К. Соколова /
«__» _____ 2026 г.

Студент группы БПИ236

_____ / С. Е. Растворов /
«__» _____ 2026 г.

Студент группы БПИ244

_____ / Л. А. Субботин /
«__» _____ 2026 г.

АННОТАЦИЯ

Руководство оператора содержит сведения о назначении программы «Разработка интеллектуального ассистента для образовательных учреждений», условиях ее выполнения, порядке запуска, основных сценариях работы и действиях пользователя в исключительных ситуациях.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. НАЗНАЧЕНИЕ ПРОГРАММЫ	4
1.1. Функциональное назначение	4
1.2. Эксплуатационное назначение	4
2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ	5
2.1. Минимальный состав аппаратных средств	5
2.2. Минимальный состав программных средств	5
2.3. Требования к персоналу (пользователю)	5
3. ВЫПОЛНЕНИЕ ПРОГРАММЫ	6
3.1. Установка программы	6
3.2. Запуск программы и работа с приложением	6
3.2.1. Контроль доступности после развёртывания	6
3.2.2. Основной сценарий — ответ на вопрос	6
3.2.3. Работа через Telegram-бот	6
3.2.4. Работа через Streamlit-клиент	7
3.2.5. Обновление корпуса документов	7
3.2.6. Проверка метрик	7
СООБЩЕНИЯ ОПЕРАТОРУ	8
Тексты сообщений оператору	8
Действия оператора в исключительных ситуациях	8

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. НАЗНАЧЕНИЕ ПРОГРАММЫ

1.1. Функциональное назначение

Программа «Разработка интеллектуального ассистента для образовательных учреждений» предназначена для интеллектуальной поддержки студентов, преподавателей и сотрудников образовательных учреждений: автоматизированные ответы на вопросы об учебных планах, регламентах, расписаниях и сервисах вуза на основе внутренних документов. Система использует архитектуру RAG (Retrieval-Augmented Generation): поиск релевантных фрагментов документов в векторной базе ChromaDB и генерация ответа с помощью LLM с явным цитированием источников.

В составе реализованных функций программа обеспечивает: приём вопроса через REST API (POST /ask), Telegram-бот или Streamlit-клиент; семантический поиск по корпусу внутренних документов с использованием эмбедингов ruBERT-tiny2; генерацию ответа через Ollama/Mistral с указанием названия документа и фрагмента-источника; деградацию до лексического резервного поиска при недоступности LLM; фоновое обновление корпуса документов через POST /index; хранение истории диалогов в PostgreSQL; сбор метрик качества через GET /metrics; аутентификацию по токenu с ролями пользователь/администратор.

1.2. Эксплуатационное назначение

Программа предназначена для интеллектуальной поддержки студентов и преподавателей НИУ ВШЭ и других вузов — ответы на вопросы об учебных планах, регламентах, сроках и процедурах на основе внутренних документов. Система базируется на RAG-архитектуре и использует локальную векторную базу (ChromaDB), языковую модель (Ollama/Mistral), эмбединги (ruBERT-tiny2) и интерфейсы Telegram/Web. Разработка выполнена командой трёх студентов НИУ ВШЭ в рамках курсового проекта по направлению 09.03.04 «Программная инженерия».

Программа предназначена для эксплуатации в составе контейнеризованного стека Docker Compose. Пользователи работают через Telegram-бот (стандартный клиент мессенджера) или Streamlit-клиент (современный браузер). Доступ к административным функциям (индексация, управление пользователями) осуществляется через REST API с токеном администратора. Штатная работа предполагает наличие доступного экземпляра Ollama с моделью Mistral; при его недоступности система автоматически переключается в лексический режим.

Программа рассчитана на эксплуатацию в пилотном режиме с аудиторией 50—100 пользователей и нагрузкой до 300 запросов в сутки. Специальной подготовки рабочего места пользователя не требуется; для работы с системой достаточно клиента Telegram или современного браузера.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. УСЛОВИЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ

2.1. Минимальный состав аппаратных средств

Программа является веб-сервисом, поэтому состав аппаратных средств определяется двумя контурами: рабочим местом пользователя и серверной инфраструктурой. Для работы с системой через Telegram-бот или Streamlit-клиент достаточно любого устройства с доступом в интернет и установленным клиентом Telegram или современным браузером. Специальные периферийные устройства не требуются.

Серверный узел должен обеспечивать одновременную работу FastAPI RAG-сервера, ChromaDB, PostgreSQL 16 и Ollama/Mistral. Минимальная конфигурация серверного узла: 4 vCPU, 16 ГБ оперативной памяти, SSD-накопитель объёмом не менее 20 ГБ для хранения ChromaDB-индекса, моделей и логов. Для ускорения генерации ответов Ollama может использовать GPU: NVIDIA-видеокарта с не менее 8 ГБ VRAM.

2.2. Минимальный состав программных средств

Для работы пользователей с системой требуется клиент Telegram (любая актуальная версия для iOS, Android или Desktop) или современный браузер с поддержкой HTML5 для доступа к Streamlit-клиенту. Дополнительное программное обеспечение на стороне пользователя не требуется.

Серверная часть разворачивается в контейнерной среде и использует следующий стек: Python 3.12, FastAPI, LangChain, ChromaDB, ruBERT-tiny2 (HuggingFace), Ollama (Mistral), PostgreSQL 16 / SQLAlchemy (asyncpg), aiogram 3.x (Telegram-бот), Streamlit (веб-клиент), Tesseract OCR, PyPDF2, Docker и Docker Compose.

В процессе эксплуатации для обновления корпуса документов должны быть доступны файлы форматов PDF, HTML, TXT и PNG/JPG. Для администрирования используются переменные окружения (.env) и командная строка Docker Compose.

2.3. Требования к персоналу (пользователю)

Для работы с программой через Telegram-бот или Streamlit-клиент от пользователя требуется уверенное владение смартфоном или компьютером и умение вводить текстовые запросы. Специальная техническая подготовка не требуется.

Пользователи с ролью администратора должны уметь работать с REST API (curl или аналогичный HTTP-клиент), запускать Docker Compose, настраивать переменные окружения и производить обновление корпуса документов через POST /index.

Обслуживающий персонал, отвечающий за развёртывание, мониторинг и резервное копирование, должен иметь базовые навыки работы с Linux, Docker, PostgreSQL и переменными окружения, а также уметь анализировать журналы контейнеров и результаты healthcheck.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ВЫПОЛНЕНИЕ ПРОГРАММЫ

3.1. Установка программы

Программа поставляется в виде набора контейнеров Docker и не требует установки на рабочую станцию оператора. Для развёртывания серверной части выполняются следующие шаги.

1. Скопировать репозиторий и перейти в корневой каталог проекта.
2. Создать файл .env на основе примера и задать переменные: DATABASE_URL (строка подключения к PostgreSQL), LLM_HOST (адрес Ollama, например http://ollama:11434), CHROMA_PATH (путь к каталогу ChromaDB), BOT_TOKEN (токен Telegram-бота), ALLOW_LLM_FALLBACK (0 в продуктивном режиме, 1 для разработки без Ollama).
3. Выполнить docker compose up -d для запуска всех сервисов.
4. Проверить готовность сервиса: GET /health должен вернуть HTTP 200.
5. Загрузить корпус документов и запустить индексацию через POST /index.

После завершения установки оператор проверяет, что ответ на тестовый вопрос через Telegram-бот или Streamlit-клиент возвращается корректно.

3.2. Запуск программы и работа с приложением

После запуска стека пользователь работает через Telegram-бот или браузер (Streamlit) и видит только те функции, которые доступны его роли.

3.2.1. Контроль доступности после развёртывания

Перед началом прикладной работы рекомендуется выполнить проверку готовности сервиса: GET /health (ожидаемый ответ HTTP 200, {«status»: «ok»}). При необходимости также проверяются GET /metrics и доступность Swagger-документации по адресу /docs. Если сервер не отвечает, необходимо проверить состояние контейнеров (docker compose ps), журналы (docker compose logs) и параметры подключения к PostgreSQL и Ollama в .env.

3.2.2. Основной сценарий — ответ на вопрос

Для получения ответа пользователь вводит вопрос на русском или английском языке в интерфейс Telegram-бота или Streamlit-клиента. Программа передаёт вопрос в POST /ask, выполняет RAG-пайплайн и возвращает ответ с текстом и перечнем документов-источников. Если Ollama недоступен, ответ формируется через лексический поиск и помечен соответствующим предупреждением.

Рекомендуемый порядок работы: ввести вопрос; ознакомиться с ответом и источниками; при необходимости задать уточняющий вопрос в рамках той же сессии для использования истории диалога.

3.2.3. Работа через Telegram-бот

Для начала работы необходимо найти бот в Telegram и отправить команду /start. Бот отобразит приветственное сообщение и краткую инструкцию. После этого можно вводить произвольные текстовые вопросы. Ответ будет содержать текст и перечень источников. Команда /help отображает справочную информацию; команда /admin доступна только пользователям с административной ролью.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

При отправке файла (PDF или изображение) с подписью-вопросом бот передаёт его на обработку и возвращает ответ на основе содержимого файла.

3.2.4. Работа через Streamlit-клиент

Для работы через Streamlit необходимо открыть в браузере адрес, по которому развёрнут клиент (по умолчанию <http://localhost:8501> для локального стенда). В диалоговой форме ввести вопрос и нажать кнопку отправки. Ответ и список источников отображаются под полем ввода. История диалога сохраняется в рамках текущей сессии браузера.

3.2.5. Обновление корпуса документов

Для обновления базы знаний администратор помещает новые документы (PDF, HTML, TXT, PNG/JPG) в каталог корпуса и выполняет POST /index с параметрами пути. Операция выполняется в фоне и не прерывает обработку текущих запросов. После завершения индексации новые документы становятся доступны для поиска. Если корпус изменился значительно, рекомендуется выполнить полную переиндексацию, предварительно очистив ChromaDB-индекс.

3.2.6. Проверка метрик

Для контроля качества работы системы доступен эндпоинт GET /metrics, возвращающий JSON с полями avg_response_time (среднее время ответа в секундах), mean_confidence (средняя оценка уверенности модели) и escalation_rate (доля запросов, переданных на ручную проверку). Эти данные используются для мониторинга и принятия решений об обновлении корпуса или смене LLM.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СООБЩЕНИЯ ОПЕРАТОРУ

3.2.6 Тексты сообщений оператору

Программа выводит сообщения оператору через интерфейс Telegram-бота, Streamlit-клиента и через HTTP-ответы API. Ниже приведены типичные сообщения для основных сценариев работы.

При успешном ответе на вопрос оператор получает текст ответа с перечнем источников в формате «Источник: [название документа], фрагмент: [цитата]». При ответе в лексическом режиме дополнительно выводится предупреждение: «Ответ сформирован в резервном режиме (лексический поиск). Основная языковая модель временно недоступна.»

При запуске Telegram-бота командой /start выводится: «Добро пожаловать! Задайте вопрос об учебных планах, регламентах или сервисах вуза.» При вводе /help выводится справочное сообщение с описанием доступных команд.

Для REST API: при успешном POST /ask — HTTP 200 с JSON-полями answer, sources, confidence, fallback; при отсутствии токена — HTTP 401, {«detail»: «Not authenticated»}; при недостаточных правах — HTTP 403, {«detail»: «Forbidden»}; при ошибке на стороне сервера — HTTP 500, {«detail»: «Internal server error»}. При GET /health в штатном режиме — HTTP 200, {«status»: «ok»}; при недоступности сервиса — HTTP 503. При POST /index — HTTP 202, {«status»: «indexing started»}; при ошибке пути — HTTP 400 с описанием причины.

3.2.6 Действия оператора в исключительных ситуациях

При получении сообщения «Ответ сформирован в резервном режиме» оператор должен убедиться в доступности Ollama (docker compose ps, docker compose logs). При необходимости перезапустить контейнер (docker compose restart ollama). После восстановления доступности Ollama сервис автоматически возвращается в основной RAG-режим без перезапуска.

При HTTP 401 на запросы к API необходимо проверить корректность токена в заголовке Authorization: Bearer . При HTTP 403 убедиться, что используется токен с соответствующей ролью: токен администратора требуется для POST /index и административных маршрутов.

При HTTP 500 или недоступности POST /ask необходимо: проверить состояние контейнеров (docker compose ps); просмотреть журналы сервера (docker compose logs server); убедиться в доступности PostgreSQL и ChromaDB; проверить переменные окружения в .env. При повторяющихся ошибках зафиксировать текст ответа и последовательность действий для передачи в поддержку.

Если GET /health возвращает HTTP 503 или не отвечает, необходимо проверить состояние всех контейнеров стека (docker compose up -d), доступность базы данных и свободное место на диске (ChromaDB-индекс и данные PostgreSQL занимают пространство на накопителе).

При сообщении «LLM временно недоступна» в Streamlit-клиенте или Telegram-боте пользователь может продолжать работу — ответы будут формироваться через лексический поиск. Оператор должен восстановить доступность Ollama и при необходимости уведомить пользователей об ожидаемых сроках восстановления штатного режима.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 34 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]